MSA 8770 Enterprise Analytics Project Milestone 1
Instructor: Dr. Saber Soleymani
Group 5 Team: Danny McIntosh, Maria Malhotra, Maya Patel, Mi Le, Tran Le

Project Name: AI-Assisted College Admissions Process

I. Business Problem and Primary User

Every year, about **3.5 to 3.8 million** students graduate from US high schools. Of these, **2.2 to 2.6 million** students will successfully matriculate to a college or university. About 1.53 million of these students will use the Common Application, the largest centralized platform for college admissions, currently being used and required by 1,140 colleges and universities, roughly **40%** of the **2,800** traditional 4-year degree-granting institutions. For the 2025–2026 admissions cycle, **1.53 million unique first-year students** used the platform to apply to college.

Each high school student will submit an average of about **6-7** applications. A typical state university will have about **30 to 50+** admissions officers processing up to **60k+** applications during the entire application season.

Primary User and Primary Human in the Loop: Admission officer, first reader in a state university that receives tens of thousands of student applications within short timelines

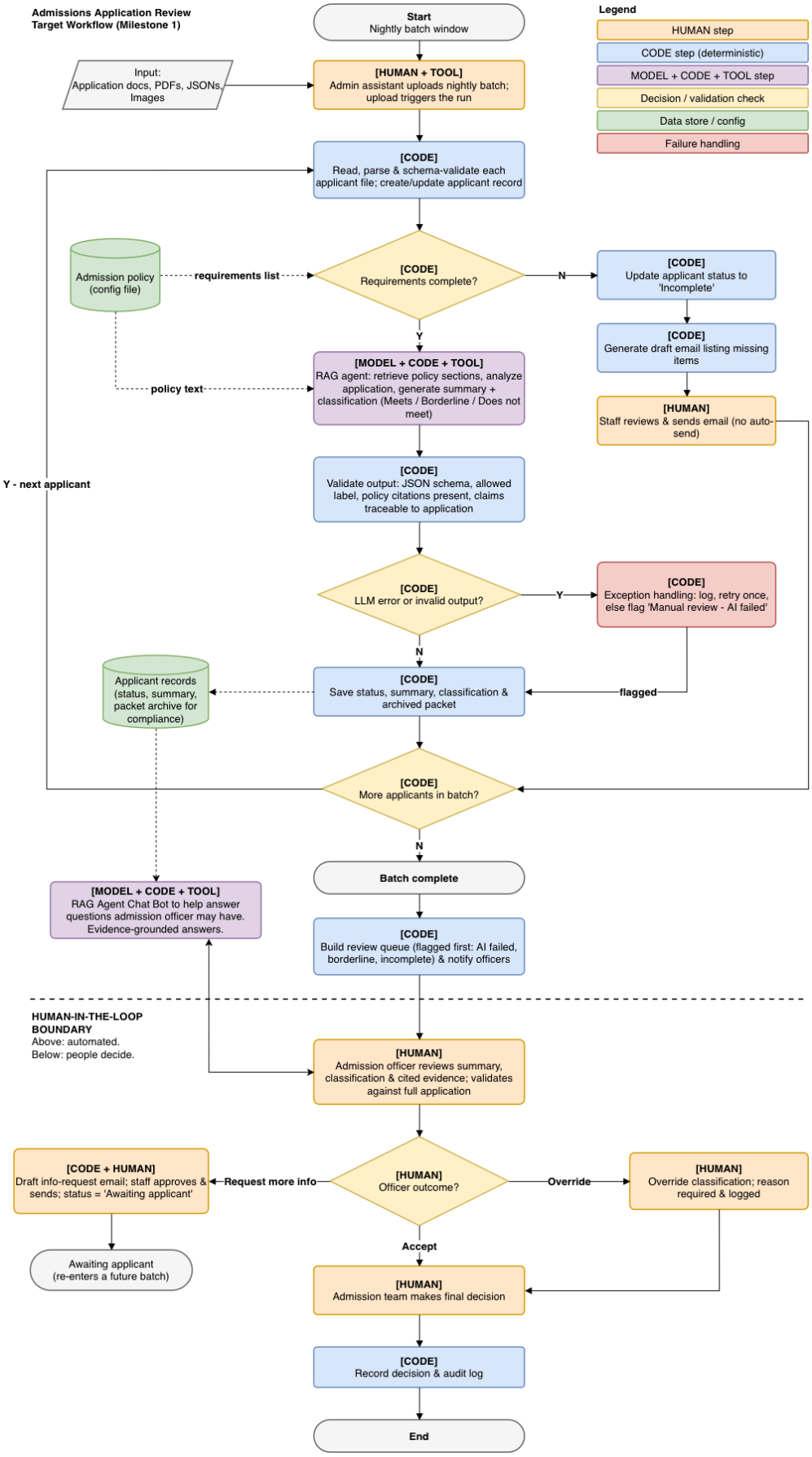
Secondary Users: 1) Admission officer second readers, other staff to handle email correspondences; 2) Dean of Admission - for final admission decisions and 3) University tech support - to handle exceptions such as LLM failures

II. Justification for AI

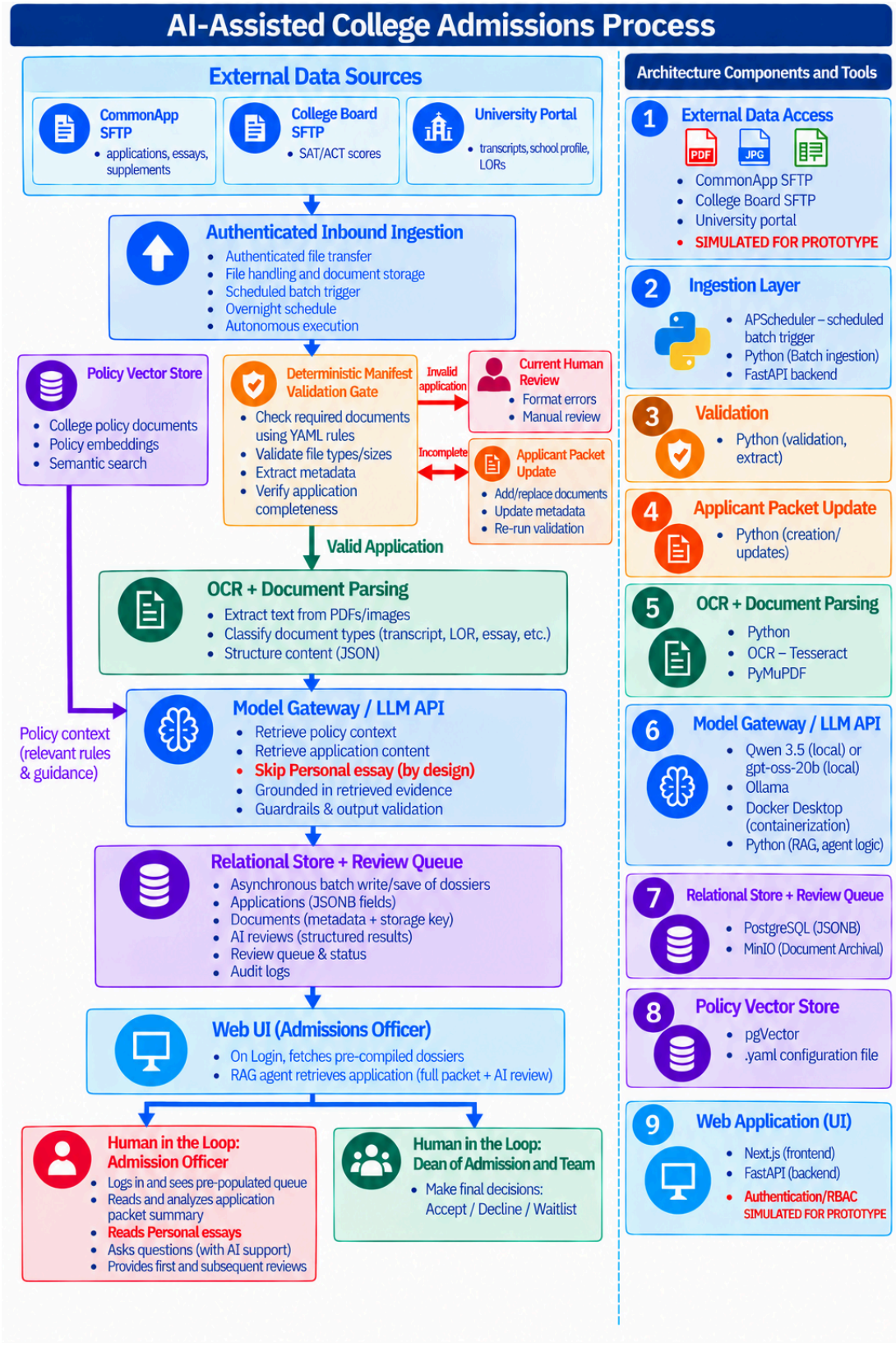
AI has become a helpful tool that can reduce the workload and save time for humans. In the school admissions case, AI is useful because the application review process is complicated and time-consuming. Applications contain many things based on each university's requirements, such as essays, recommendation letters, transcripts, SAT scores, and other documents that vary in format and wording. The proposed model, using the RAG agent / and UI chatbot, can help admissions officers in many ways. It can read these materials, retrieve relevant admissions policy, summarize the application, and suggest whether an applicant meets the requirements, is borderline, or does not meet the requirements. It also provides citations so officers can check the evidence. There are some steps that do not need AI and should stay deterministic. Code should match uploaded files to the correct applicant, check file types and required fields, and compare. More details about this can be found in the next sections of this document.

However, AI can make mistakes, and a wrong AI summary or classification could affect an applicant's opportunity and result in significant consequences. The model should therefore never make the final admissions decision. The system is purposely designed so that the personal essay will not be evaluated by AI and will only be read and evaluated by humans. A personal essay contains the applicant's personal experiences, emotions, voice, and individual story, and AI should not be responsible for interpreting the full meaning and emotional context of that essay. Admissions officers also emphasize the importance of personally reading applicants' essays, so that the human part is preserved in this admission process. Hence the implementation of human-in-the-loop: an admissions officer reviews the full application, checks the cited evidence, accepts or overrides the AI suggestion with a recorded reason, and makes all final decisions.

III. Target Workflow


Admissions Application Review
Target Workflow (Milestone 1)
Start
Nightly batch window
Legend
HUMAN step
CODE step (deterministic)
MODEL + CODE + TOOL step
Decision / validation check
Data store / config
Failure handling
Input:
Application docs, PDFs, JSONs, Images
[HUMAN + TOOL]
Admin assistant uploads nightly batch; upload triggers the run
[CODE]
Read, parse & schema-validate each applicant file; create/update applicant record
Admission policy (config file)
requirements list
[CODE]
Requirements complete?
N
[CODE]
Update applicant status to 'Incomplete'
[CODE]
Generate draft email listing missing items
[HUMAN]
Staff reviews & sends email (no auto-send)
Y
policy text
[MODEL + CODE + TOOL]
RAG agent: retrieve policy sections, analyze application, generate summary + classification (Meets / Borderline / Does not meet)
Y - next applicant
[CODE]
Validate output: JSON schema, allowed label, policy citations present, claims traceable to application
[CODE]
LLM error or invalid output?
Y
[CODE]
Exception handling: log, retry once, else flag 'Manual review - AI failed'
N
Applicant records (status, summary, packet archive for compliance)
[CODE]
Save status, summary, classification & archived packet
flagged
[CODE]
More applicants in batch?
N
Batch complete
[MODEL + CODE + TOOL]
RAG Agent Chat Bot to help answer questions admission officer may have. Evidence-grounded answers.
[CODE]
Build review queue (flagged first: AI failed, borderline, incomplete) & notify officers
HUMAN-IN-THE-LOOP BOUNDARY
Above: automated.
Below: people decide.
[HUMAN]
Admission officer reviews summary, classification & cited evidence; validates against full application
[CODE + HUMAN]
Draft info-request email; staff approves & sends; status = 'Awaiting applicant'
Request more info
[HUMAN]
Officer outcome?
Override
[HUMAN]
Override classification; reason required & logged
Awaiting applicant (re-enters a future batch)
Accept
[HUMAN]
Admission team makes final decision
[CODE]
Record decision & audit log
End

IV. Architecture Plan



The system architecture implements an asynchronous batch evaluation pipeline backed by relational and vector storage, isolated document parsing and LLM inference, and a human-in-the-loop review console. External sources (simulated via Common App, College Board SFTP, and an admissions portal) feed raw application profiles, standardized test scores, and transcripts. An authenticated Python/FastAPI ingestion service uses an automated scheduler (APScheduler) to run off-peak overnight intake cycles without manual uploads. Inbound packets immediately encounter a deterministic manifest validation gate written in Python, which checks required checklist items, file formats, and completeness against YAML rules. This deterministic gate rejects incomplete files and routes them to update workflows or manual queues, conserving downstream model compute and preventing unnecessary token costs.

Valid applications proceed to OCR and document parsing (PyMuPDF/Tesseract), which extracts tabular coursework, credits, and grades from multi-format transcripts into standardized JSON schemas. The Model Gateway then invokes an LLM to evaluate academic discrepancies against institutional rules and synthesize an evaluation dossier. By design, personal essays are excluded from AI processing and reserved strictly for human evaluation to prevent algorithmic bias. Structured dossiers are written asynchronously to PostgreSQL (status = 'READY_FOR_REVIEW'), while raw files are archived in MinIO. PostgreSQL decouples background batch ingestion from user activity. When an admissions officer logs into the web console (Next.js/Streamlit), the application fetches pre-compiled dossiers instantly, enabling reviewers to verify policy citations, read personal essays, and record final decisions without blocking on background processing.

The initial model strategy deploys an open-source model (gpt-oss-20b or Qwen 2.5) hosted locally via Ollama and Docker Desktop within the institution's private infrastructure. Local execution ensures compliance with FERPA data-privacy standards by keeping sensitive student records off public third-party APIs. If local hardware limits extraction fidelity during testing, the gateway can route de-identified payloads to hosted enterprise frontier models under zero-retention agreements. The planned retrieval strategy grounds evaluations in institutional policies, such as GPA minimums, English proficiency waivers, and transfer credit rules. A version-controlled configuration file (policies.yaml) injects discrete rules directly into the prompt context for predictable adherence. For unstructured handbook sections, text chunks (300–500 tokens) are indexed in PostgreSQL using pgvector, allowing semantic queries that attach explicit handbook citations to flagged anomalies.

Two distinct trust boundaries secure the pipeline. Trust Boundary 1 sits between external data feeds and the internal ingestion service, treating all inbound multi-part PDFs and JSON payloads as untrusted input subject to MIME verification, file size limits, and manifest checks. Trust Boundary 2 surrounds the internal secure environment, enclosing PostgreSQL, MinIO, and the review console. All sensitive data (including student PII, official transcripts, and disciplinary remarks) resides strictly inside this perimeter. Access requires authenticated role based controls, and all officer overrides and admission decisions are logged to immutable audit tables.

V. Data and Feasibility

The project will use a fully synthetic applicant database to simulate a university admissions portal. Each applicant will have a unique Applicant ID, structured database record, and supporting PDF documents such as transcripts, recommendation letters, test scores, personal statements, and exception-related documentation. The dataset will include complete applications and intentional edge cases, including missing or duplicate documents, conflicting

information, unreadable attachments, international requirements, and cases requiring human review. A ground-truth file will record expected application statuses, missing documents, applicable policies, and escalation outcomes for evaluation.

All applicant information is fictional and contains no real student records or personally identifiable information, avoiding FERPA, confidentiality, and consent concerns during development. A production system would require appropriate authentication, role-based access, encryption, retention controls, and audit logging.

The prototype will be developed in Python using PostgreSQL and MinIO for data storage, Tesseract or PyMuPDF and OCR for document processing, an LLM API for extraction and summarization, pgVector and a .yaml configuration file for policy retrieval, and JSON Schema for output validation.

The primary feasibility risk is maintaining realistic and internally consistent synthetic data while incorporating meaningful edge cases. Documents and expected results will therefore be generated from the same master record and validated through automated checks of Applicant IDs, dates, GPA values, document inventories, and ground-truth statuses. If full portal integration is not feasible, a structured folder upload and local database will simulate the university portal.

VI. Design Decision Records

DDR 1: Deterministic Validation Before AI Processing	
Decision	Use rule-based validation to check application completeness, required documents, file formats, and explicit admissions requirements before AI analysis. Invalid or incomplete cases are routed to human review instead.
Rationale	Objective requirements can be checked without an LLM involved. This reduces unnecessary model calls (reducing costs), catches data-quality issues early, and separates deterministic rules from tasks requiring judgment for better processing traceability.
Alternative	Use the LLM to determine application completeness and whether requirements are satisfied.
Trade-off	Deterministic checks produce consistent and testable results, but require maintaining a strict set of rules and may not be able to handle ambiguous cases or policy exceptions without human review, increasing human workload.
Evidence	In Milestone 2, the team will test complete, incomplete, invalid-format, and exception cases against expected validation outcomes.

DDR 2: Retrieval-Augmented Generation for Admissions Policy Context and Chatbot	
Decision	Use RAG with a pgVector to retrieve relevant admissions rules and policies for AI-assisted application review and to retrieve application information for the admission officer Q&A chatbot.
Rationale	Admissions policies can vary and application details are specific on a per case

	basis. RAG gives both the review workflow and chatbot access to set policy and applicant-specific information and supports evidence-grounded responses without retraining the model.
Alternative	Rely on the LLM's pretrained general knowledge; place policies in a fixed prompt; or provide a chatbot without retrieval.
Trade-off	RAG would help with grounding, consistency, and information traceability across both the review analysis and chatbot, but adds retrieval and storage complexity. Poor retrieval can still lead to incomplete, inaccurate, or misleading responses.
Evidence	Week 3 materials noted RAG as a way to provide changing external knowledge without retraining the entire model. In Milestone 2, the team will test retrieval relevance, review accuracy, chatbot answer correctness, response hallucinations, and citation accuracy using representative sample admissions cases and questions.

DDR 3: Human-in-the-Loop Admissions Decisions	
Decision	Use AI to organize application information, generate structured review analysis, and answer grounded questions, but admissions officers retain responsibility for final accept, reject, or waitlist decisions. All system routes should end with human review.
Rationale	Admissions decisions involve subjective and contextual factors with impactful outcomes. Human review allows officers to verify AI outputs, examine original materials, and apply judgment before making the final decision. Fully automated decisions could have detrimental consequences and raise ethical concerns.
Alternative	Allow the AI system to make final admissions decisions without needing mandatory human approval.
Trade-off	Human oversight improves accountability and reduces the risk of harmful AI errors (better governance), but limits automation and adds review time (lower efficiency). Full automation may be faster, but increases governance and decision-risk issues.
Evidence	The proposed architecture requires human officers to review through the admissions console before a final decision is recorded. Later milestones will test that final outcomes/actions cannot be completed without authorized human approval.

DDR 4: Batch Processing Cadence: Nightly vs. End-of-Cycle	
Decision	Process newly submitted or updated applications in nightly batches rather than waiting until the application deadline to process the full applicant pool at once.
Rationale	Nightly processing spreads computational workload over time, identifies missing

	or invalid documents earlier, and makes completed applications available for review sooner. In theory, it would reduce the time needed to review applications manually but not interfere with decision timeline (i.e. early admissions, rolling admissions, etc.).
Alternative	Process the full applicant pool in one batch after the application deadline.
Trade-off	Nightly batches prevent workload spikes at the end of application deadlines and surface issues earlier so that applicants and officers have more chances for corrections. However, updated applications may need reprocessing which increases costs. End-of-cycle processing is simpler and the system would be more direct, but it would delay error detection and review readiness.
Evidence	To be measured in later milestones using batch size, processing time, reprocessing frequency, review-readiness time, and exception rates.

VII. Team Plan

For **Milestone 1,** we all worked together in true collaboration, but for purposes of the milestone requirements, these were the roles, which could be rotated for every milestone:

1. Maria Malhotra: Project Lead & Solution Architect
2. Mi Le: AI Workflow & Orchestration
3. Tran Le: Integration & Human Oversight
4. Danny McIntosh: Production & Platform
5. Maya Patel: Governance, Security, & Risk

The team uses a shared Google Document to organize project documentation, code, diagrams, research, meeting notes, and completed work. Members meet in person at least once a week to review progress, make design decisions, and assign tasks, with additional Zoom meetings as needed. Before each milestone submission, all members review the final deliverable for consistency across documentation, diagrams, data, and system design.

AI was used to generate synthetic applicant metadata as well as to assist with creating colored diagrams. All applicant data is fictional and contains no real student records or personally identifiable information. The team validated the synthetic data through manual sample reviews and automated checks for required fields, unique Applicant IDs, document consistency, file counts, and ground-truth statuses. All AI-assisted artifacts were reviewed and revised by team members before inclusion.